Module 8 Portfolio Project - Option 2:

Face Detection and Privacy

Alexander Ricciardi

Colorado State University Global

CSC515: Foundations of Computer Vision

Dr. Bingdong Li

November 8, 2026

Module 8 Portfolio Project - Option 2:

Face Detection and Privacy

This paper explains how the Python program `Module8_Portfolio_Option2.py` detects human faces and blurs their eyes as a privacy treatment in three color photographs. One photograph is a close portrait taken indoors, while the second shows six people standing outdoors. The third shows a cat and is used to examine whether the human-face classifier selects a nonhuman subject. These images present different problems for the program. The group faces and eyes occupy fewer pixels than those in the portrait, and regions that are not faces can also be selected. Finding a face does not establish that its eyes have been located accurately. The program compares face-detection settings, extracts clean face regions, and aligns them before searching for the eyes again and applying Gaussian blur, which reduces image detail by averaging neighboring pixel values with different weights. This explanation examines the methods, repeated experiments, and resulting images to determine where detection improves and where errors remain. Face detection, eye-center accuracy, and blur coverage are measured separately because finding and covering an eye are different results. Please see the Appendix section for the program's images and terminal output.

Preparing the Images

The first photograph is a 2,615 × 3,269 pixel NASA portrait showing one primarily front-facing person under indoor lighting (NASA, 1998). The second is a 3,000 × 1,996 pixel NASA photograph of six primarily front-facing people outdoors, including their full bodies (NASA, 2011). Their small apparent face size provides the distant-face example, although the camera distance is unknown. The third photograph shows a cat at 451 × 300 pixels (scikit-image, n.d.). It enters the same human-face detector as the other images, allowing incorrect selections on a

nonhuman subject to be examined. The scenes differ in brightness and color, but the subjects and surroundings also change, so illumination cannot be examined separately. In `load_and_prepare()`, `cv2.imread()` reads each original color photograph. The program then uses `cv2.cvtColor()` to make a grayscale copy, replacing the three color values at each pixel with a single brightness value between 0 and 255 (Dey, 2025). These copies are used for the first detection test without resizing. The original color photographs are kept unchanged for the face crops and final output images.

Face Detection and Repeated Experiments

OpenCV uses pretrained Haar cascades to look for brightness patterns associated with faces (Viola & Jones, 2001). Each selected region is returned as a rectangle, called a candidate. Some candidates contain faces, while others are incorrect selections. The program outlines every candidate in red, including the incorrect ones. For the first test, `haarcascade_frontalface_default.xml` runs with `scaleFactor=1.1`, `minNeighbors=5`, and a minimum detection size of 24 × 24 pixels. The `scaleFactor` setting controls the search across different sizes. The `minNeighbors` setting determines how much support a selection needs from nearby detections (OpenCV, n.d.-a). This test detects the face in the portrait along with eight regions that are not faces. The group photograph produces seven boxes: six around faces and one around an incorrect region. Although no faces are missed, the extra boxes show that the results still contain errors. Switching to `haarcascade_frontalface_alt.xml`, with the other settings unchanged, eliminates those nine extra boxes and keeps the seven correct face detections.

The next test keeps the alternate cascade and limits the longest image edge to 1,000 pixels. Resizing preserves the proportions of each photograph and leaves the full scene in view. The resulting images measure 800 × 1,000 pixels for the portrait and 1,000 × 665 pixels for the

group. The detector again finds the seven faces without selecting any additional regions. This configuration, `03_alt_scaled`, is used for the remaining processing. Further tests examine changes in image contrast and the support needed from neighboring detections. Global histogram equalization redistributes brightness values across the whole image. In this test, the portrait face is detected, but two faces in the group are missed. Contrast limited adaptive histogram equalization (CLAHE) works on smaller image regions and limits contrast amplification. Using `clipLimit=2.0` and an 8 × 8 tile grid, the CLAHE test detects the seven faces (OpenCV, n.d.-c). A separate test raises `minNeighbors` to 7 from its initial value of 5. This setting demands more support from neighboring detections, and one group face is missed. None of these three tests produces an extra box. Both contrast methods are applied independently to the same resized grayscale input; neither method processes the output of the other. The results of all six experiments, including missed detections, are recorded in `face_measurements.csv`; see Table A1 in the Appendix section for the recorded data. For these photographs, adding more processing or requiring stronger detection support does not necessarily produce a better result.

Extracting and Aligning the Faces

The selected rectangles are mapped back to the original photographs to extract clean color face crops. Each crop also has a working copy with a longest edge of 300 pixels for the first eye search. This enlarges the smaller group faces, but it cannot recover detail missing from the original photograph. The `haarcascade_eye.xml` classifier first returns candidate eye rectangles. The program then compares their spacing, relative size, and position within the face to select a plausible pair. The reference marks used for scoring do not guide this selection. When the search fails, it tries equalization, an eyeglasses cascade, and separate left and right searches. Several search settings change together, so any improvement cannot be attributed to the separate

regions alone. The first estimated eye pair supplies the angle, scale, and position needed to align the face. Rotation, uniform scaling, and translation place it on a 320 × 320 canvas, with eye targets at (96, 121.6) and (224, 121.6) (OpenCV, n.d.-b). The classifier then searches for the eyes again on the aligned image. All seven faces receive treatment, although two group faces need three and five final search attempts. Separately marked eye locations retain eye-line angles between -1.77° and +2.05° after alignment. The estimated control points can be placed on a horizontal line even when they do not coincide with the actual eye centers. Therefore, the transformation does not establish perfectly horizontal eyes.

Blurring the Eyes

Gaussian blur calculates a weighted average of neighboring pixel values, giving greater weight to pixels closer to the center of the calculation. This reduces fine detail in the eye regions (OpenCV, n.d.-d). In `blur_eyes()`, the initial treatment extends each side of an eye box by 35% of its width or height. This padding includes some surrounding area, such as eyelids and eye corners. Sigma determines the spread of the Gaussian weights and is initially 10% of the larger eye-box dimension. The final treatment increases padding to 60% and sigma to 45% of that dimension; both versions use a minimum sigma of two pixels. Figure 1 compares the aligned faces with their initial and final blur results. The wider treatment covers more surrounding area, while the stronger blur also removes more detail there. Since padding and sigma change together, their individual effects cannot be separated from this comparison. Code Excerpt 1 shows that both treatments begin with the same aligned face. The final result is not calculated by applying more blur to the initial blurred image.

Code Excerpt 1

Initial and Revised Eye Blur in process_faces()

```
weak, weak_mask, weak_settings = blur_eyes(
```

```
    face["aligned_bgr"], final_pair, face["valid_mask"],
    WEAK_SIGMA_FRACTION, INITIAL_EYE_PADDING)
# Apply the stronger Gaussian blur with the wider final treatment area
strong, mask, strong_settings = blur_eyes(
    face["aligned_bgr"], final_pair, face["valid_mask"], FINAL_SIGMA_FRACTION)
```

Note. Both calls use the same aligned face. The final call uses the default 60% padding.

Figure 1

Initial and Revised Eye Blur on the Seven Detected Faces

Note. Each set of three images shows the aligned face followed by the initial and final blur results. Photographs from NASA (1998, 2011).

The blurred eye regions are then placed back into the original photographs using the inverse transformation. Only pixels selected by the treatment mask are replaced. Comparing the remaining pixels with the originals confirms that all of their values are unchanged. The final

photographs in Figure 2 contain 125,130 changed pixels in the portrait and 23,920 in the group image. No pixels change in the cat image because it receives no treatment. More pixels are affected in the portrait, where the face is much larger. However, the number of changed pixels cannot determine whether a person is still identifiable.

Figure 2

Final Outputs for the Three Photographs

Note. The portrait and group photograph show the final eye blur. The cat photograph remains unchanged. Sources: NASA (1998, 2011) and scikit-image (n.d.).

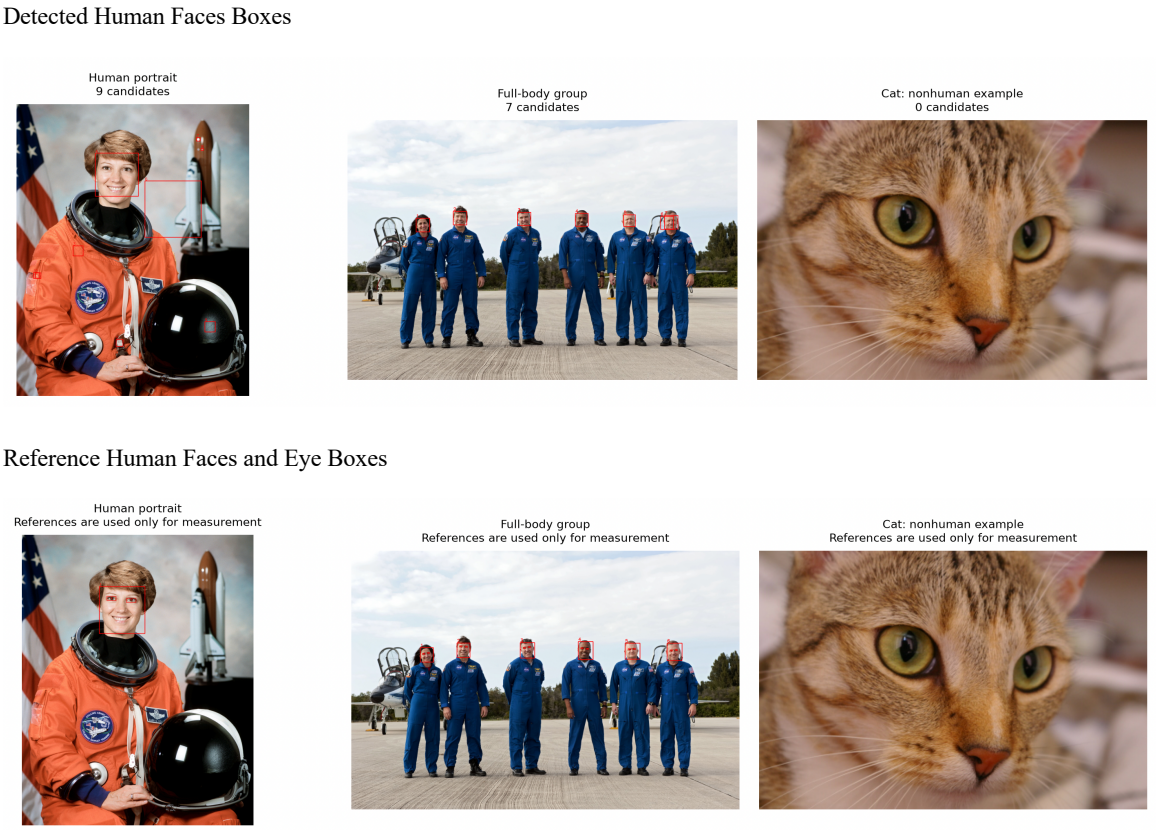
Observed Results and Accuracy

Seven approximate reference face boxes marked on the original photographs are used to evaluate face detection; see Figure 3. Intersection-over-union (IoU) divides the area shared by a detected box and its reference box by their union; $IoU = Area\ of\ Intersection\ /\ Area\ of\ Union$. This union contains the area inside either box, counting the overlap once. A match requires IoU of at least 0.50. With the selected settings, the portrait face and all six group faces are matched, with no extra detections or missed faces. This corresponds to seven true positives (TP), zero false positives (FP), and zero false negatives (FN) across the two human photographs. Other calculations performed are, the precision metric divides the number of matching detections by the total number of detections; $precision = TP\ /\ (TP + FP)$. The

recall metric divides the number of reference faces found by the total number of reference faces. These calculations output a 1.00 value for both measures on each human photograph. Note that a value of 1.00 means that all detected faces matched a reference face and all reference faces were detected in these photographs. F1 combines precision and recall, $F1 = 2 \times (precision \times recall) / (precision + recall)$, and it also outputs a value of 1.00 for each human photograph. These results show that the reference boxes are matched by the detected face boxes at the required IoU threshold on the supplied images; they do not establish how well the classifier will perform on other photographs.

Figure 3

Original Photograph with Reference and Detected Eyes and Faces Boxes



Note: The figure showcases the original photographs with reference and detected faces and eyes boxes.

The eye-center measurements, which are calculated in the `measure_results()` function, show a different result. An eye center passes when its distance from the marked reference center is no more than 10% of the separation between the reference eyes, with a minimum tolerance of two pixels. Both portrait centers pass, compared with four of the twelve group eye centers. Both centers pass on just one of the six group faces, giving totals of six correct centers out of fourteen and two complete pairs out of seven. The small group faces leave little room for error because the tolerance also depends on the distance between their eyes. For example, one group face has a tolerance of about four pixels, while its center errors are 6.02 and 4.66 pixels. Neither center passes, even though the final mask covers both marked eye regions.

The eye-region coverage measurements, also found in the `measure_results()` function, help explain why an eye can be blurred even when its detected center is inaccurate. The initial masks fully cover twelve of the fourteen marked eye regions. After widening the padding, the final masks cover all fourteen. The additional area can include an eye that lies away from the center of its detection window, but it does not move that center to a more accurate position. Eight centers therefore still fail the localization test. All six face experiments return no human-face candidates for the cat, and its pixels remain unchanged. Precision is reported as N/A because there are no detections to evaluate, while recall is N/A because the photograph has no reference human faces. These undefined values should not be interpreted as 100% detection accuracy.

Conclusion and Limitations

For these photographs, the selected face cascade finds all seven human faces without extra boxes, and the final treatment covers all fourteen marked eye regions. However, the eye-center errors remain even after increasing the blurred area. Covering every marked eye does not mean that its center was located accurately. The reference face boxes and eye centers also have

limitations because they were marked by visual inspection. A small shift in an eye-center mark could change whether a group eye center passes the tolerance. All three images were examined when choosing the settings, so none is an independent test on an unseen photograph. Another limitation concerns the privacy treatment itself. Blurring the eyes leaves the rest of the face, clothing, and other identifying details visible. The program does not test whether a person can still be identified from the final image. Therefore, the measured coverage confirms where the treatment was applied, but it cannot establish guaranteed anonymization.

References

Dey, S. (2025). *Image processing and computer vision masterclass with Python: Learn advanced image processing techniques, computer vision fundamentals, and applications* (2nd ed.). BPB Publications.

NASA. (1998, October 30). *S99-00858* [Photograph]. NASA Image and Video Library. https://images.nasa.gov/details/S99-00858

NASA. (2011, February 20). *KSC-2011-1515* [Photograph]. NASA Image and Video Library. https://images.nasa.gov/details/KSC-2011-1515

OpenCV. (n.d.-a). *Cascade classifier*. OpenCV 4.13.0 documentation. https://docs.opencv.org/4.13.0/db/d28/tutorial_cascade_classifier.html

OpenCV. (n.d.-b). *Geometric transformations of images*. OpenCV 4.13.0 documentation. https://docs.opencv.org/4.13.0/da/d6e/tutorial_py_geometric_transformations.html

OpenCV. (n.d.-c). *Histograms - 2: Histogram equalization*. OpenCV 4.13.0 documentation. https://docs.opencv.org/4.13.0/d5/daf/tutorial_py_histogram_equalization.html

OpenCV. (n.d.-d). *Smoothing images*. OpenCV 4.13.0 documentation. https://docs.opencv.org/4.13.0/d4/d13/tutorial_py_filtering.html

scikit-image. (n.d.). *skimage.data.chelsea*. scikit-image documentation. https://scikit-image.org/docs/stable/api/skimage.data.html#skimage.data.chelsea

Viola, P., & Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. In *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (Vol. 1, pp. 511–518). IEEE. https://doi.org/10.1109/CVPR.2001.990517

Appendix

This section is reserved for screenshots from the program execution, including the terminal explanation, original images, and the Matplotlib views, as well as a table storing the face detection data results of the six experiments.

Figure A1

Original Images

Note: Image 1 shows NASA astronaut Eileen Collins, used as the close-up human portrait. Image 2 shows the six astronauts of Space Shuttle Discovery's STS-133 crew at Kennedy Space Center. From left to right, they are mission specialists Nicole Stott, Michael Barratt, Steve Bowen, and Alvin Drew; pilot Eric Boe; and commander Steve Lindsey. The image is used for the multiple human with their full bodies visible requirement. Image 3 shows Chelsea the cat, photographed by Stefan van der Walt; the image is used as the nonhuman example used to evaluate whether the human-face detector produces false detections.

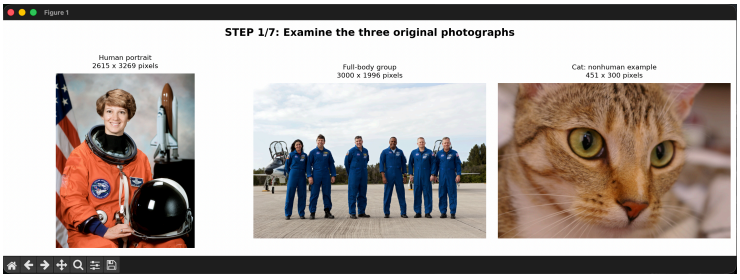
Figure A2

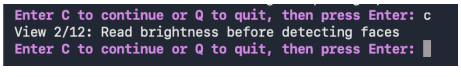
Image outputs - Blur Treated Images

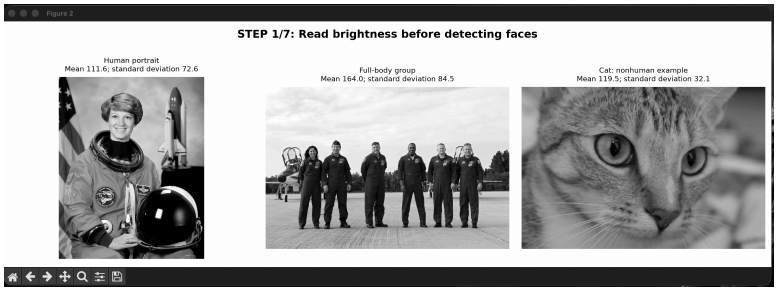
Note: The images showcase that only the human faces were recognized as human faces, and their eyes were blurred for privacy purposes.

Figure A3

Terminal walkthrough







```
View 2/12: Read brightness before detecting faces
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 2/7: Detect faces with the initial settings
================================================================================

The initial classifier is haarcascade_frontalface_default.xml. This is a pretrained Haar
cascade, not a person-identification model. It examines intensity patterns at different
positions and sizes. A returned box is a face candidate, not proof that the enclosed
region is a human face.

The baseline uses scaleFactor=1.1, minNeighbors=5, and minSize=(24, 24). scaleFactor
sets the spacing between search scales. minNeighbors sets the support required when
nearby candidates are grouped. minSize excludes smaller search windows. None of these
settings is a calibrated probability of correctness.

Look at the red boxes and compare them with the original scenes. All returned candidates
remain visible, including background mistakes and duplicate boxes. The separate
reference view shows approximate visual annotations made from the original photographs.
Those marks never enter the detector or choose an eye pair.

A detected face is matched to at most one reference face. Intersection-over-union
divides the overlapping area by the combined area of the boxes. Here, a match requires
at least 0.50. The matching routine first maximizes the number of valid one-to-one
matches, then their total overlap. An extra box is a false positive, and an unmatched
reference face is a false negative.

Across the three baseline images, this run produced 7 matched faces, 9 extra boxes, and
0 missed reference faces. Read the tables in the next step to see where they occurred.

Saved teaching view: figures/03_initial_faces.png
Saved teaching view: figures/04_reference_marks.png

View 3/12: Initial face candidates at native resolution
Enter C to continue or Q to quit, then press Enter:
```

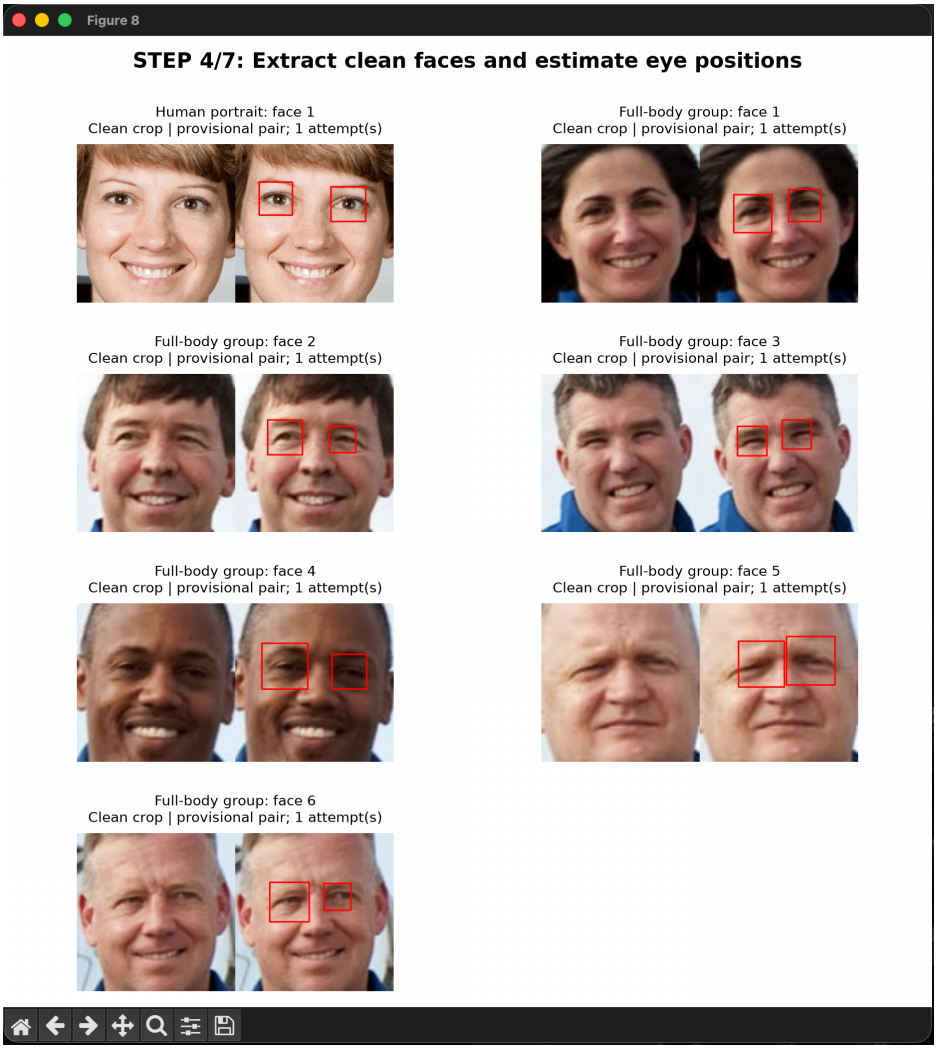
```
Enter C to continue or Q to quit, then press Enter: c
View 4/12: Visual reference marks, not detector outputs
Enter C to continue or Q to quit, then press Enter:
```

```
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 3/7: Compare the revisions
================================================================================

Each revision starts from the stored original grayscale image. The second experiment
changes only the face model to haarcascade_frontalface_alt.xml. The third retains that
model but caps the longer scene edge at 1,000 pixels, preserving its proportions. This
changes the sampling grid and the effective original-image size represented by the
24-pixel minimum search window.

The fourth experiment applies global histogram equalization to the scaled input. The
fifth instead uses CLAHE with clipLimit=2.0 and an 8 x 8 tile grid. Both are compared
with the plain scaled image, not chained together. The sixth returns to plain grayscale
and increases minNeighbors from 5 to 7. A stricter setting can remove a false positive,
but it can also remove a real face.

Human portrait.
    Experiment              TP   FP   FN   Precision   Recall       F1
    01_default_native        1    8    0       0.111    1.000    0.200
    02_alt_native            1    0    0       1.000    1.000    1.000
    03_alt_scaled            1    0    0       1.000    1.000    1.000
    04_alt_equalized         1    0    0       1.000    1.000    1.000
    05_alt_clahe             1    0    0       1.000    1.000    1.000
    06_alt_stricter          1    0    0       1.000    1.000    1.000

Full-body group.
    Experiment              TP   FP   FN   Precision   Recall       F1
    01_default_native        6    1    0       0.857    1.000    0.923
    02_alt_native            6    0    0       1.000    1.000    1.000
    03_alt_scaled            6    0    0       1.000    1.000    1.000
    04_alt_equalized         4    0    2       1.000    0.667    0.800
    05_alt_clahe             6    0    0       1.000    1.000    1.000
    06_alt_stricter          5    0    1       1.000    0.833    0.909

Cat: nonhuman example.
    Experiment              TP   FP   FN   Precision   Recall       F1
    01_default_native        0    0    0         N/A      N/A      N/A
    02_alt_native            0    0    0         N/A      N/A      N/A
    03_alt_scaled            0    0    0         N/A      N/A      N/A
    04_alt_equalized         0    0    0         N/A      N/A      N/A
    05_alt_clahe             0    0    0         N/A      N/A      N/A
    06_alt_stricter          0    0    0         N/A      N/A      N/A

Precision is TP/(TP+FP), recall is TP/(TP+FN), and F1 is 2TP/(2TP+FP+FN). N/A means the
denominator is zero. The cat has no reference human faces, so its recall is N/A, not
100%. A count of zero is a correct outcome on this one negative image, not a
population-wide specificity estimate.

The selected configuration is 03_alt_scaled for all three photographs. It was chosen
during development and is fixed in the script. The program does not pick a different
model for each image or use an expected face count to suppress mistakes. Contrast
changes and the stricter setting remain in the record even when their results are worse.

Saved teaching view: figures/05_comparison_eileen_collins.png
Saved teaching view: figures/05_comparison_sts133_arrival.png
Saved teaching view: figures/05_comparison_chelsea.png

View 5/12: Compare human portrait
Enter C to continue or Q to quit, then press Enter:
```

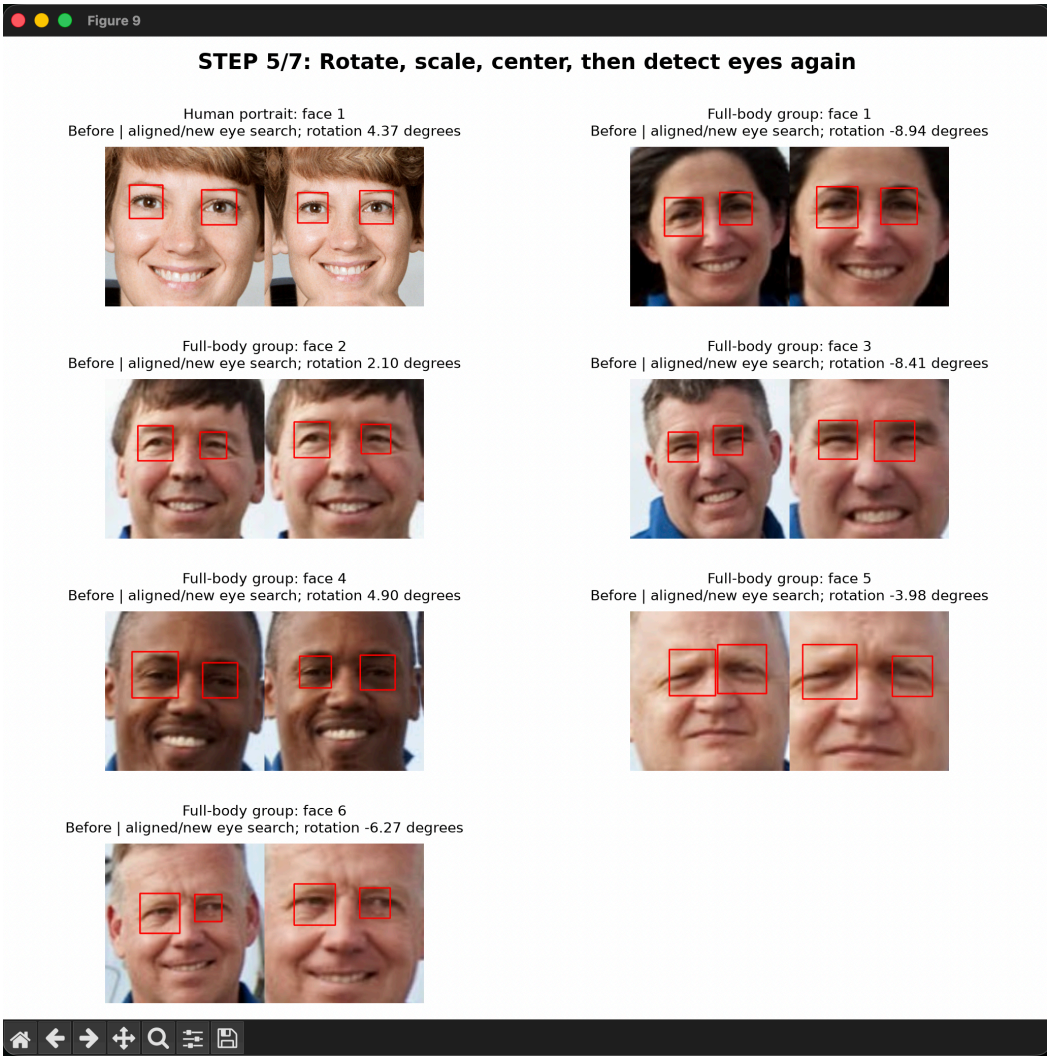
```
Enter C to continue or Q to quit, then press Enter: c
View 6/12: Compare full-body group
Enter C to continue or Q to quit, then press Enter:
```

```
Enter C to continue or Q to quit, then press Enter: c
View 7/12: Compare cat: nonhuman example
Enter C to continue or Q to quit, then press Enter:
```

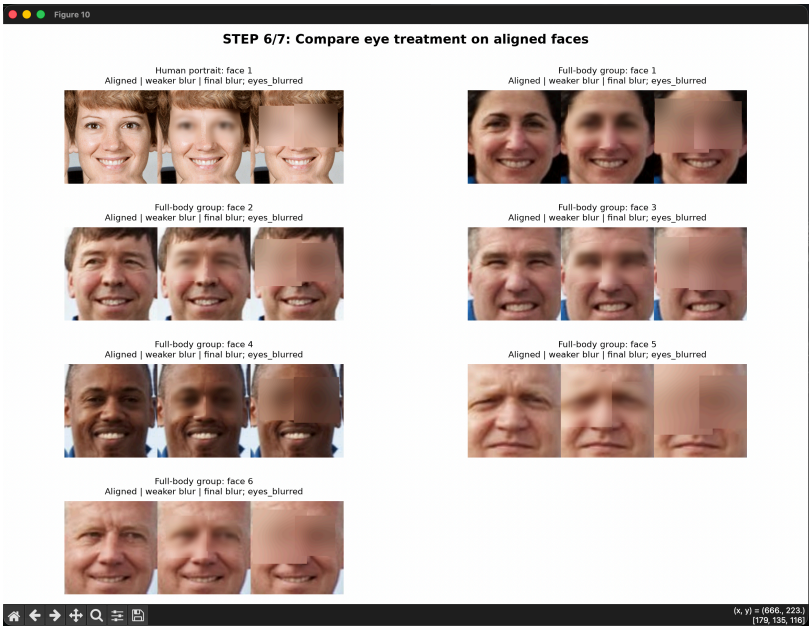
```
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 4/7: Extract faces and estimate eye positions
================================================================================

The red boxes were drawn on display copies. Face extraction uses the untouched original
color photographs instead. The selected detector boxes are mapped back with the actual
horizontal and vertical resize factors, then used to slice the original arrays. NumPy
uses [y, x], while each OpenCV rectangle is (x, y, width, height).

A separate face copy is resized to a maximum edge of 300 pixels for the provisional eye
search. Enlarging a small face changes how the cascade samples it; it does not restore
detail that the camera did not record. The clean original crop is kept for the later
geometric transformation.

The first eye search uses haarcascade_eye.xml on the upper 70% of the crop. Overlapping
proposals are filtered, and candidate pairs are checked for plausible spacing, height,
size, and placement on opposite sides of the face. These checks can reject a nose or
brow proposal, but they do not establish that a pair is correct.

When no pair passes, the program tries equalization and the eye-tree-eyeglasses cascade.
The last attempts search the upper left and right halves separately with a finer scale
step and lower grouping requirement. That last revision changes more than one setting,
so its effect cannot be attributed to the region alone. Every attempted candidate list
is saved. A missing pair remains unresolved.

Saved teaching view: figures/06_extracted_faces.png

View 8/12: Extract clean faces and estimate eye positions
Enter C to continue or Q to quit, then press Enter:
```

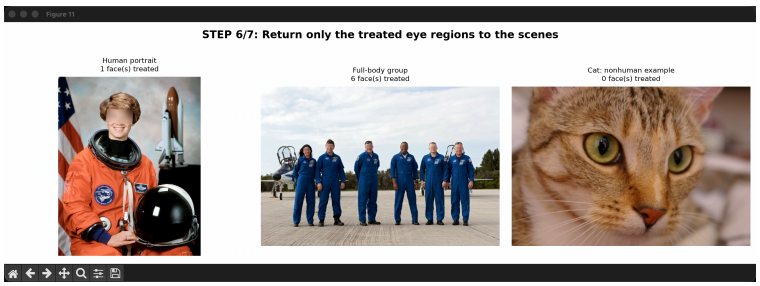


```
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 5/7: Align faces and find the eyes again
================================================================================

A first eye estimate is needed before the face can be oriented. align_face() calculates
the angle between those centers with atan2(), then uses their distance to calculate one
uniform scale. The translation moves their midpoint to the center of the target eye
line. This performs rotation, scaling, and centering; it is more than a square resize.

The common canvas is 320 x 320 pixels. Its target eye centers are (96, 121.6) and (224,
121.6), which puts them at 30% and 70% of the width and 38% of the height. warpAffine()
samples the original crop using the calculated matrix. Reflected pixels can fill the
display border, but a separate validity mask prevents those artificial border pixels
from becoming treatment data.

Now look at the eye boxes on the aligned faces. They come from a new cascade search
after the transformation. They are not the original boxes moved into place. The
provisional estimates, final detections, transformation matrix, inverse matrix, and all
retry settings are retained separately.

The two estimated control points map to their targets within a small numerical
tolerance. That checks the transformation mathematics, not the true eye locations. The
program also transforms the independent reference eye marks and measures their remaining
tilt. Approximate cascade-window centers can leave real eyes slightly misaligned even
when the control-point check passes.

Saved teaching view: figures/07_alignment.png

View 9/12: Rotate, scale, center, then detect eyes again
Enter C to continue or Q to quit, then press Enter:
```



```
Enter C to continue or Q to quit, then press Enter: c

================================================================================
STEP 6/7: Blur the eyes and restore the scenes
================================================================================

The eye treatment starts from each aligned color crop. The first margin is 35% of the
detected eye-box width and height. The final margin increases to 60% on every face to
cover more of the eye boundaries. Gaussian blur replaces pixels only within those padded
regions. The comparison also shows a weaker sigma of 10% of the eye-box size beside a
stronger final sigma of 45%. The minimum sigma is 2 pixels. The actual odd kernel and
sigma are saved for each eye. Both padding and strength change, so this comparison does
not isolate blur strength.

Look for remaining eye detail in the weaker version before examining the final one. The
final blur is deliberately strong, which also removes some surrounding detail inside the
padded regions. A less visible seam would not necessarily mean better privacy. These
pictures show the treatment; they are not a re-identification test.

The inverse transformation maps the blurred pixels and their binary mask back to the
original crop. Color uses linear interpolation, while the mask uses nearest-neighbor
interpolation. Only pixels selected by that mask are replaced in the full scene. The
program checks exact equality outside the mask rather than trusting that the two images
look similar.

Human portrait: 1 face(s) received eye treatment; 125,130 pixels changed; outside-mask
pixels unchanged: True.

Full-body group: 6 face(s) received eye treatment; 23,920 pixels changed; outside-mask
pixels unchanged: True.

Cat: nonhuman example: 0 face(s) received eye treatment; 0 pixels changed; outside-mask
pixels unchanged: True.

There is no animal-specific branch that forces the cat to remain unchanged. It enters
the same human-face pipeline. When the selected detector returns no face candidates, the
treatment mask is empty and the original decoded pixels remain unchanged. Any false face
detection would still be reported.

Saved teaching view: figures/08_blur_strengths.png
Saved teaching view: figures/09_privacy_scenes.png

View 10/12: Compare eye treatment on aligned faces
Enter C to continue or Q to quit, then press Enter:
```






```
Enter C to continue or Q to quit, then press Enter: c
View 11/12: Return only the treated eye regions to the scenes
Enter C to continue or Q to quit, then press Enter:
```

Note: The figure showcases the terminal output and the images generated with the Matplotlib library.

Table AI

Face Detection Experiments Results

scene	experiment	cascade	preprocessing	detector_width	detector_height	scale_factor	min_neighbors	min_size	tp	fp	fn	precision	recall	f1	elapsed_seconds
eileen_collins	01_default_native	haarcascade_frontalface_default.xml	gray	2615	3269	1.1	5	24	1	8	0	0.1111111111111110	1.0	0.2	1.023199292132630
eileen_collins	02_alt_native	haarcascade_frontalface_alt.xml	gray	2615	3269	1.1	5	24	1	0	0	1.0	1.0	1.0	1.1118002079892900
eileen_collins	03_alt_scaled	haarcascade_frontalface_alt.xml	gray	800	1000	1.1	5	24	1	0	0	1.0	1.0	1.0	0.14342400012537800
eileen_collins	04_alt_equalized	haarcascade_frontalface_alt.xml	equalized	800	1000	1.1	5	24	1	0	0	1.0	1.0	1.0	0.1431107500102370
eileen_collins	05_alt_clahe	haarcascade_frontalface_alt.xml	clahe	800	1000	1.1	5	24	1	0	0	1.0	1.0	1.0	0.16175383306108400
eileen_collins	06_alt_stricter	haarcascade_frontalface_alt.xml	gray	800	1000	1.1	7	24	1	0	0	1.0	1.0	1.0	0.13519008411094500
sts133_arrival	01_default_native	haarcascade_frontalface_default.xml	gray	3000	1996	1.1	5	24	6	1	0	0.8571428571428570	1.0	0.9230769230769230	0.7001439579762520
sts133_arrival	02_alt_native	haarcascade_frontalface_alt.xml	gray	3000	1996	1.1	5	24	6	0	0	1.0	1.0	1.0	0.662254833849147
sts133_arrival	03_alt_scaled	haarcascade_frontalface_alt.xml	gray	1000	665	1.1	5	24	6	0	0	1.0	1.0	1.0	0.09087858302518730
sts133_arrival	04_alt_equalized	haarcascade_frontalface_alt.xml	equalized	1000	665	1.1	5	24	4	0	2	1.0	0.6666666666666670	0.8	0.13144066696986600
sts133_arrival	05_alt_clahe	haarcascade_frontalface_alt.xml	clahe	1000	665	1.1	5	24	6	0	0	1.0	1.0	1.0	0.12104391609318600
sts133_arrival	06_alt_stricter	haarcascade_frontalface_alt.xml	gray	1000	665	1.1	7	24	5	0	1	1.0	0.8333333333333330	0.9090909090909090	0.08941137488000100
chelsea	01_default_native	haarcascade_frontalface_default.xml	gray	451	300	1.1	5	24	0	0	0				0.02961529209278520
chelsea	02_alt_native	haarcascade_frontalface_alt.xml	gray	451	300	1.1	5	24	0	0	0				0.027331499848514800
chelsea	03_alt_scaled	haarcascade_frontalface_alt.xml	gray	451	300	1.1	5	24	0	0	0				0.02613837504759430
chelsea	04_alt_equalized	haarcascade_frontalface_alt.xml	equalized	451	300	1.1	5	24	0	0	0				0.02665412495844070
chelsea	05_alt_clahe	haarcascade_frontalface_alt.xml	clahe	451	300	1.1	5	24	0	0	0				0.026855416828766500
chelsea	06_alt_stricter	haarcascade_frontalface_alt.xml	gray	451	300	1.1	7	24	0	0	0				0.02625841717235740

Note: The table showcases the results stored in face_measurements.csv. The file stores the results of the six face-detection experiments performed on the three photographs: a single-person portrait, a six-person group photograph, and a cat image.